

GameMath.Clamp called

590

===== GAME MAP 3 * 4 =====

1 2 4 0

4 4 3 3

2 4 0 0

Legend: 0=Grass 1=Water 2=Mountain 3=Forest 4=Dungeon

Tile Count:

Grass : 3

Water : 1

Mountain : 2

Forest : 2

Dungeon : 4

Printing global value: 1

Printing local value: 10

Calling Nested Namespace:

Playing: sword_clash

```
<terminated> (exit value: 0) Day 11.exe [C:\C++ Application] D:\C++\Day 11\
Name: Aragorn
Health: 100
Level: 10
Type: Player
```

```
Name: Orc
Health: 60
Level: 5
Type: Enemy
```

```
Name: HealthPotion
Health: 0
Level: 1
Type: Item
```

```
Physics Clamp: Physics Clamp called
50
Physics Lerp: Physics Lerp called
259
```

```
GameMath Clamp: GameMath Clamp called
50
GameMath Lerp: Gamemath Lerp called
4035
Physics Clamp called
20
GameMath Clamp called
590
```

```
===== GAME MAP 3 * 4 =====
```

```
1 2 4 0
4 4 3 3
2 4 0 0
```

```
Legend: 0=Grass 1=Water 2=Mountain 3=Forest 4=Dungeon
```

```
Tile Count:
Grass : 3
Water : 1
Mountain : 2
Forest : 2
```

```

cout<<"==== GAME MAP "<<R<<" * "<<C<<" "<<"===="<<endl;
int cnt1 = 0, cnt2 = 0, cnt3 = 0, cnt4 = 0, cnt5 = 0;
for(int i = 0; i < R; i++){
    for(int j = 0; j < C; j++){
        populateCount(cnt1, cnt2, cnt3, cnt4, cnt5, grid[i][j]);
        cout<<grid[i][j]<<" ";
    }
    cout<<endl;
}
cout<<"Legend: 0=Grass 1=Water 2=Mountain 3=Forest 4=Dungeon"<<endl;
cout<<endl;
cout<<"Tile Count:"<<endl;
cout<<"Grass : "<<cnt1<<endl;
cout<<"Water : "<<cnt2<<endl;
cout<<"Mountain : "<<cnt3<<endl;
cout<<"Forest : "<<cnt4<<endl;
cout<<"Dungeon : "<<cnt5<<endl;
int level = 10;
cout<<"Printing global value: "<<::level<<endl;
cout<<"Printing local value: "<<level<<endl;
cout<<"Calling Nested Namespace: "<<endl;
cout<<endl;
Engine::Audio::playSound("sword_clash");
return 0;
}

```

```

    }
    }
}

int main() {
    Entity player, enemy, item;
    player.setName("Aragorn").setHealth(100).setLevel(10).setType("Player");
    enemy.setName("Orc").setHealth(60).setLevel(5).setType("Enemy");
    item.setName("HealthPotion").setHealth(0).setLevel(1).setType("Item");
    player.displayInfo();
    enemy.displayInfo();
    item.displayInfo();
    cout<<"Physics Clamp: ";
    cout<<Physics::clamp(30, 50, 100);
    cout<<endl;
    cout<<"Physics Lerp: ";
    cout<<Physics::lerp(3, 7, 64);
    cout<<endl<<endl;
    cout<<"GameMath Clamp: ";
    cout<<GameMath::clamp(-2, 50, 100);
    cout<<endl;
    cout<<"GameMath Lerp: ";
    cout<<GameMath::lerp(3, 66, 64);
    cout<<endl;
    {
        using namespace Physics;
        cout<<clamp(10, 20, 3000);
    }
    cout<<endl;
    {
        using namespace GameMath;
        cout<<clamp(8000, 20, 590);
    }
    cout<<endl;
    int R, C;
    R = 3;
    C = 4;
    int** grid = new int*[R];

    for(int i = 0; i < R; i++){
        grid[i] = new int[C];
        for(int j = 0; j < C; j++){
            grid[i][j] = (rand() % 5);
        }
    }

    cout<<"==== GAME MAP "<<R<<" * "<<C<<" "<<"===="<<endl;
    int cnt1 = 0, cnt2 = 0, cnt3 = 0, cnt4 = 0, cnt5 = 0;
    for(int i = 0; i < R; i++){
        for(int j = 0; j < C; j++){

```

```

    if(max < min) return 0;
    cout<<"Physics Clamp called"<<endl;
    if(val < min) return min;
    if(val > max) return max;
    return val;
}

double lerp(double a, double b, double t){
    cout<<"Physics Lerp called"<<endl;
    return a + t * (b-a);
}
}

namespace GameMath {
    int clamp(int val, int min, int max){
        if(max < min) return 0;
        cout<<"GameMath Clamp called"<<endl;
        if(val < min) return min;
        if(val > max) return max;
        return val;
    }

    double lerp(double a, double b, double t){
        cout<<"Gamemath Lerp called"<<endl;
        return a + t * (b-a);
    }
}

void populateCount(int& cnt1,int& cnt2,int& cnt3,int& cnt4,int& cnt5, int& gridval){
    if(gridval == 0){
        cnt1++;
    }
    else if(gridval == 1){
        cnt2++;
    }
    else if(gridval == 2){
        cnt3++;
    }
    else if(gridval == 3){
        cnt4++;
    }
    else{
        cnt5++;
    }
}

int level = 1;
namespace Engine{
    namespace Audio{
        void playSound(string name){
            cout<<"Playing: "<<name<<endl;

```

```

#include <iostream>
using namespace std;
class Entity{
private:
    string name;
    int health;
    int level;
    string type;
public:
    Entity& setName(const string& name){
        this->name = name;
        return *this;
    }
    Entity& setHealth(int health){
        this->health = health;
        return *this;
    }
    Entity& setLevel(int level){
        this->level = level;
        return *this;
    }
    Entity& setType(const string& type){
        this->type = type;
        return *this;
    }

    string getName() const{
        return this->name;
    }
    int getHealth() const{
        return this->health;
    }
    int getLevel() const{
        return this->level;
    }
    string getType() const{
        return this->type;
    }
    void displayInfo() const{
        cout<<"Name: "<<this->name<<endl;
        cout<<"Health: "<<this->health<<endl;
        cout<<"Level: "<<this->level<<endl;
        cout<<"Type: "<<this->type<<endl;

        cout<<endl<<endl;
    }
};

namespace Physics {
    double clamp(double val, double min, double max){

```

```

};

int main(){
    Patient p1;
    Patient p2(102, "Gurunath", 52, "General", "O+");
    Patient p3(101, "Sakshi");
    Patient* patients[4];
    for(int i=0; i<4; i++){
        int id, age;
        string name, ward, bg;
        cout<<"Enter Details for the patient: "<<endl;
        cout<<"Enter ID: ";
        cin>>id;
        cout<<"Enter Name: ";
        cin>>name;
        cout<<"Enter Age: ";
        cin>>age;
        cout<<"Enter Ward: ";
        cin>>ward;
        cout<<"Enter Blood Group: ";
        cin>>bg;
        patients[i] = new Patient(id, name, age, ward, bg);
    }
    for (int i = 0; i < 4; i++){
        patients[i]->displayRecord();
    }
    patients[1]->transferWard("ICU");
    for (int i = 0; i < 4; i++){
        delete patients[i];
    }
    return 0;
}

```

```

#include<iostream>
#include<string>
using namespace std;
class Patient{
    int patientId;
    string name;
    int age;
    string ward;
    const string bloodGroup;
public:
    Patient():patientId(0),name("Unknown"),age(0),ward("General"),bloodGroup("O+"){
        cout<<"[Constructor] Default patient registered." << endl;
    }
    Patient(int id,const string& name):patientId(id),name(name),age(0),ward("Emergency"),bloodGroup("O+"){
        cout<<"[Constructor] Emergency: "<<name<< endl;
    }
    Patient(int id,const string& name,int age,const string& ward,const string& bg):
        patientId(id),name(name),age(age),ward(ward),bloodGroup(bg){
        cout<<"[Constructor] Full admission: "<<name<<endl;
    }
    ~Patient(){
        cout<<"[Destructor] Patient: "<<name<<" discharged."<<endl;
    }
    void displayRecord()const{
        cout<<"Patient Record: "<<endl;
        cout<<"ID : "<<patientId<<endl;
        cout<<"Name : "<<name<<endl;
        cout<<"Age : "<<age<<endl;
        cout<<"Ward : "<<ward<<endl;
        cout<<"Blood Group : "<<bloodGroup<<endl;
    }
    void transferWard(const string& newWard){
        cout<<"Ward transfer : "<<name<<"->"<<newWard<<endl;
        ward=newWard;
    }
};

int main(){
    Patient p1;
    Patient p2(102,"Gurunath",52,"General","O+");
    Patient p3(101,"Sakshi");
    Patient* patients[4];
    for(int i=0;i<4;i++){
        int id,age;
        string name,ward,bg;
        cout<<"Enter Details for the patient: "<<endl;
        cout<<"Enter ID: ";
        cin>>id;
        cout<<"Enter Name: ";

```


Patient Record:

ID : 1003

Name : Raj

Age : 62

Ward : Cardiology

Blood Group : B-

Patient Record:

ID : 1004

Name : Shruti

Age : 42

Ward : General

Blood Group : O+

Ward transfer : Viddhi->ICU

[Destructor] Patient: Meera discharged.

[Destructor] Patient: Viddhi discharged.

[Destructor] Patient: Raj discharged.

[Destructor] Patient: Shruti discharged.

[Destructor] Patient: Sakshi discharged.

[Destructor] Patient: Gurunath discharged.

[Destructor] Patient: Unknown discharged.

Enter Ward: Cardiology

Enter Blood Group: B-

[Constructor] Full admission: Raj

Enter Details for the patient:

Enter ID: 1004

Enter Name: Shruti

Enter Age: 42

Enter Ward: General

Enter Blood Group: O+

[Constructor] Full admission: Shruti

Patient Record:

ID : 1001

Name : Meera

Age : 34

Ward : Cardiology

Blood Group : B+

Patient Record:

ID : 1002

Name : Viddhi

Age : 25

Ward : General

[Constructor] Default patient registered.

[Constructor] Full admission: Gurunath

[Constructor] Emergency: Sakshi

Enter Details for the patient:

Enter ID: 1001

Enter Name: Meera

Enter Age: 34

Enter Ward: Cardiology

Enter Blood Group: B+

[Constructor] Full admission: Meera

Enter Details for the patient:

Enter ID: 1002

Enter Name: Viddhi

Enter Age: 25

Enter Ward: General

Enter Blood Group: O+

[Constructor] Full admission: Viddhi

Enter Details for the patient:

Enter ID: 1003

Enter Name: Raj

Enter Age: 62

```

double cost = qty * unitPrice;
double tax = cost * taxRate;
return cost + tax;
}

double applyDiscount(double price, double discountPercent = 10.0) {
    double discountAmount = price * discountPercent / 100;
    return price - discountAmount;
}

int main() {
    int n = 5;
    Product items[5];
    cout << "==== ENTER PRODUCT DETAILS =====> << endl;
    for (int i = 0; i < n; i++) {
        cout << "\nProduct number " << (i + 1) << ":" << endl;
        items[i].acceptDetails();
    }
    cout << "\n===== INVENTORY REPORT =====> << endl;
    cout << "ID\tName\tPrice\tQty\tTotalValue" << endl;
    for (int i = 0; i < n; i++) {
        items[i].displayDetails();
        cout << endl;
    }
    int highestIndex = 0;
    for (int i = 1; i < n; i++) {
        if (items[i].totalValue() > items[highestIndex].totalValue()) {
            highestIndex = i;
        }
    }
    cout << "\nHighest Value Product: " << items[highestIndex].getName()
        << " (Rs. " << items[highestIndex].totalValue() << ")" << endl;
    int threshold;
    cout << "\nEnter low stock threshold: ";
    cin >> threshold;

    cout << "Low Stock Products: ";
    for (int i = 0; i < n; i++) {
        if (items[i].isLowStock(threshold)) {
            cout << items[i].getName() << " ";
        }
    }
    cout << endl;
    cout << "\n===== FUNCTION OVERLOADING DEMO =====> << endl;

    int qty1 = 100;
    double price1 = 25.50;
    cout << "reorderCost (int, double) = " << reorderCost(qty1, price1) << endl;
    double qty2 = 12.5;
    cout << "reorderCost (double, double) = " << reorderCost(qty2, price1) << endl;
}

```

```

7 #include <iostream>
8 #include <string>
9 using namespace std;
10 class Product {
11 private:
12     int productId;
13     string name;
14     double price;
15     int quantity;
16 public:
17     void acceptDetails() {
18         cout << "Enter Product ID: ";
19         cin >> productId;
20         cout << "Enter Name: ";
21         cin >> name;
22         cout << "Enter Price: ";
23         cin >> price;
24         cout << "Enter Quantity: ";
25         cin >> quantity;
26     }
27     void displayDetails() const {
28         cout << productId << "\t" << name << "\t" << price
29             << "\t" << quantity << "\t" << totalValue();
30     }
31     double totalValue() const {
32         return price * quantity;
33     }
34     bool isLowStock(int threshold) const {
35         if (quantity < threshold) {
36             return true;
37         } else {
38             return false;
39         }
40     }
41     string getName() const {
42         return name;
43     }
44 };
45 double reorderCost(int qty, double unitPrice) {
46     double cost = qty * unitPrice;
47     return cost;
48 }
49 double reorderCost(double qty, double unitPrice) {
50     double cost = qty * unitPrice;
51     return cost;
52 }
53 double reorderCost(int qty, double unitPrice, double taxRate) {
54     double cost = qty * unitPrice;

```

Enter Product ID: Enter Name: Enter Price: Enter Quantity:

Product number 5:

Enter Product ID: Enter Name: Enter Price: Enter Quantity:

===== INVENTORY REPORT =====

ID	Name	Price	Qty	TotalValue	
1001	Widget-A	250	120	30000	
1002	Bolt-B	15.5	8	124	
1003	Widget	0	0	0	
0		0	401773333	0	
1313815392			6.95168e-310	1275067104	8.86386e-301

Highest Value Product: Widget-A (Rs. 30000)

Enter low stock threshold: Low Stock Products: Bolt-B Widget

===== FUNCTION OVERLOADING DEMO =====

reorderCost (int, double) = 2550

reorderCost (double, double) = 318.75

reorderCost (int, double, double) = 3009

applyDiscount with default 10% = 450

applyDiscount with 25% = 375

Product number 1:
Enter Product ID: 1001
Enter Name: Widget-A
Enter Price: 250.00
Enter Quantity: 120

Product number 2:
Enter Product ID: 1002
Enter Name: Bolt-B
Enter Price: 15.50
Enter Quantity: 8

Product number 3:
Enter Product ID: 1003
Enter Name: Widget -B
Enter Price: Enter Quantity:

Product number 4:
Enter Product ID: Enter Name: Enter Price: Enter Quantity:

Product number 5:
Enter Product ID: Enter Name: Enter Price: Enter Quantity:
----- INVENTORY REPORT -----